Page No. | Date

Encoders. (reverse operation of decoder)

↳ 2^n (or less) input lines and n output lines

example 8 X 3 encoder.

Input								Output		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

$x = D_4 + D_5 + D_6 + D_7$

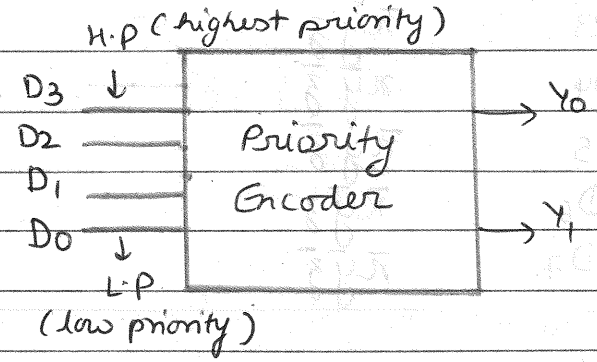
$y = D_2 + D_3 + D_6 + D_7$

$z = D_1 + D_3 + D_5 + D_7$

Priority Encoder. (special type of encoder)

↳ priority is given to input lines

↳ if two or more lines are 1 at same time, then input line with highest priority will be considered.

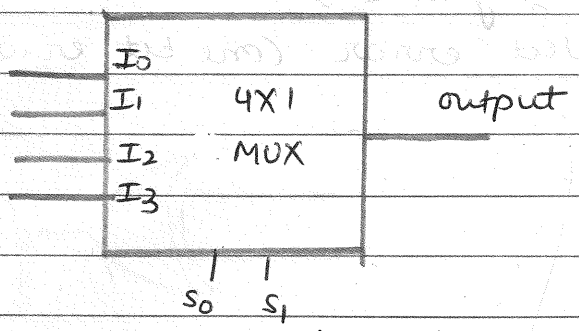


Page No.
Date

DeMultiplexer (MUX)

↳ one input line, 2^n output lines

↳ selection of particular line is controlled by select lines (n select lines)



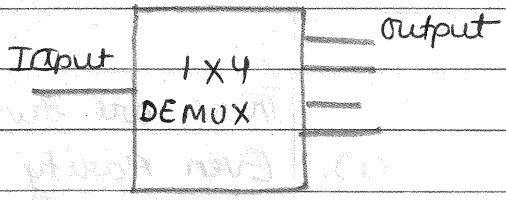
MUX

S_1	S_0	output
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

↑

multiplexer. (reverse of MUX)

↳ 2^n input lines, 1 output line



Multiplier. (shift + Add)

↳ multiply two binary number.

↳ partial product are generated for each digit

↳ all partial product are added to produce final value.

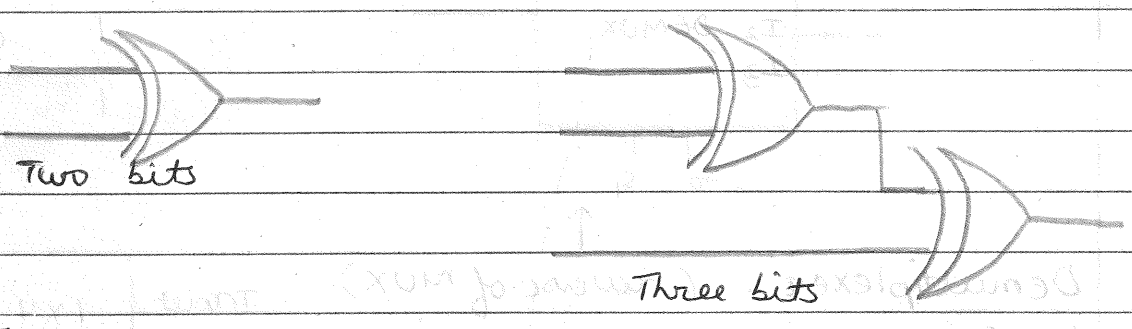
↳ each successive partial product is shifted one position to the left before adding all partial product.

```
  1 1
X 1 1
-----
  1 1     = partial product 1
1 1       = partial product 2
-------
1 0 0 1   = sum
```

Page No.
Date

Parity Generator (Ex-OR or XOR)

↳ generates parity bits (an extra bit which is added in binary message such that total no of 1's in message can be either even or odd according to parity used)



There are two types of parity generator

(i) Even Parity Generator : even no of 1's

(ii) ODD parity Generator : odd no of 1's

Four bit message	Even Parity	ODD parity
0000	0	1
0001	1	0
0010	1	0
0011	0	1

Parity checker

↳ checks parity at receiver end.

↳ checks for 1 bit error.

↳ output of parity checker is denoted by PEC (Parity Error checker) - if error, indicates 1
if no error indicates 0.

Page No.
Date

Code Converters (XOR + AND)
↳ conversion circuit must be inserted b/w two system if each uses different codes for same information.
↳ it makes two system compatible even though each uses different code.
↳ eg: binary to gray, gray to binary

Binary Comparators (XNOR)
↳ digital comparators or logic comparators.
↳ used to test value represented by one binary word is greater than, less than or equal to value represented by another binary word.

(i) Equality Comparators: used for passwords etc matching (ie only equality)
(ii) Magnitude Comparators: makes basis for decision making in logic circuits. (for less than or greater than). Any logical problem can be reduced to one or more yes/no decision based on pair of compared values.

Hazards (undesirable effect)
↳ caused by deficiency in system or external influence.

(i) Static hazard, one input changes, output changes at one time when it should not.
→ is usually due to NOT gate within logic

Elimination → (i) insert another delay to circuit
or
(ii) insert more logic to counteract effects.

(ii) Dynamic Hazard.

↳ output changes more than one as a result of single input change.

↳ occur when there are different routes to output. ie if each route has different delay, then there is possibility for changing output values.

Dynamic hazard cannot occur if all static hazard are removed.

(iii) Function Hazard.

↳ non solvable hazard which occur when more than one input changes at same time.

↳ cannot be logically eliminated as problem lies with actual specification of circuit.

Race condition.

↳ result depends on sequence in which event happen.

↳ race condition occur when system/device is designed assuming particular sequence of events without taking steps to ensure it.

Page No.
Date

- Sequential circuit

Asynchronous sequential circuits
↳ whose outputs depend upon order in which its input variable changes and can be affected at any instant of time.

Synchronous sequential circuit
↳ have a clock signal as one of their input to force change in output at discrete instance of time.
(changes at 0 or 1, rising or falling edge of clock)

Memory element used in clocked sequential circuit is called Flip-flop.

Flip flop
↳ capable of storing one bit of information.
↳ has two output → one normal value and another complement value of bit stored.

SR Flip Flop (Set - Reset)

Clock	S	R	Q	$\overline{Q}$
1	0	0	unchanged	unchanged
1	0	1	0	1
1	1	0	1	0
1	1	1	undefined	undefined.

D flip flop (Delay)

Clock	D	Q	Q_{n+1}
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Page No.
Date

JK flipflop

clock	J	K	Q	Q_{n+1}
1	0	0	x	Q_n
1	0	1	x	0
1	1	0	x	1
1	1	1	x	$\overline{Q_n}$

T flipflop

clock	T	Q_{n+1}
1	0	Q_n
1	1	$\overline{Q_n}$

Registers.

is a group of flipflop with each flipflop capable of storing one bit of information. 4 bit register → 4 FF.

Shift register. (constructed by D flipflop)

L is used to assemble and store information arriving from a serial source

L n-bit shift register consist of n-flipflop and gates control shift operation.

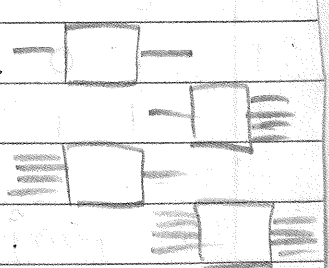
Four types of shift register

(i) SISO (serial-in-serial-out)

(ii) SIPO (serial-in-parallel-out)

(iii) PISO (parallel-in-serial-out)

(iv) PIPO (parallel-in-parallel-out).



Page No.
Date

Counters.

↳ no. of times a particular event has happened.

↳ up counter → increment in value

↳ down counter → decrement in value.

Counters.

- Asynchronous counter (TFlipFlop)
  - ripple counter (mod 2^n)
    - decoding error is there.
    - count = $2^n - 1$
- Synchronous counter. (clock)
  - ring counter (mod n)
  - twisted ring counter (mod 2n)

Page No.
Date

Some important points.

(i)

Logic Gates	no. of NAND gates	no. of NOR gates
NOT	1	1
AND	2	3
OR	3	2
Ex OR	4	5
Ex NOR	5	4

(ii) $A \oplus B \oplus C$ → total nand gates = (9)

(iii) $\bar{A} \oplus B = A \oplus \bar{B} = A \odot B$

$\bar{A} \odot B = A \odot \bar{B} = A \oplus B$

(iv) EXOR → parity checker & generator

Ex OR → addition, subtraction

Ex NOR → comparator.

(v) NAND & NOR doesnot follow associative law but follow commutative law.

(vi) OR, AND, EXOR, Ex NOR follow associative & commutative law.

(v) No of boolean func^n formed over n- boolean variable $= 2^{2^n}$

ex: 4 boolean variable $= 2^{2^4} = 2^{16} = 65536$

(vi) Half substractor no of NAND / NOR gates required = (5)

Full substractor no of NAND / NOR gates required = (9)

↳ 2 half substractor + 1 OR gate + 1 NOT gate,

Page No.
Date

eg: MUX	to be implement MUX	required no of MUX
2:1	4:1	3
4:1	16:1	4+1 = 5
4:1	64:1	21
8:1	64:1	9
8:1	256:1	37

eg: $\frac{64}{4} = 16$ (3rd one solved)

$\frac{16}{4} = 4$

$\frac{4}{4} = 1$

4:1 → 64:1

16+4+1 = (21)

21 MUX required to form 64:1 from 4:1.

eg: MUX is universal logic circuit

(vi) Decoder with enable input can func^n as demux.

eg: Decoder	to be implemented Decoder	required.
2:4	4:16	
2:4	3:8	
4:(16)	8:(256)	same division method.

→ Look ahead carry adder

no of AND gates required in carry circuit for 'n' bit look ahead carry adder is

$N_{AND} = \frac{n(n+1)}{2}$ $\quad N_{OR} = n$

Page No.
Date

(i) Comparator

$A > B \Rightarrow A\bar{B}$

$A < B \Rightarrow \bar{A}B$

$A = B \Rightarrow AB + \bar{A}\bar{B}$

(ii) Flip Flop circuit is known as "Bistable Multivibrator" or "Latch".

(iii) JK is called universal flipflop.
bcz D, SR, T flipflop can be derived from it

(iv) D flipflop delay → exactly one cycle of CLK.

(v) Race cond^n arises in "Asynchronous circuit".

(vi) Master Slave FF is formed by SR FF followed by T-FF.

(vii) In shift register → "serial input" decides what goes to leftmost flipflop during shift.

(viii) All shift register are made of JK FF.
PIPO → made of D-flipflop.
↳ is not a shift register

(ix) SIPO → temporal code into spacial code.

Page No.
Date

	Asychronous counter	Synchronous counter
(I)	different clock are applied	(I) same clock is applied.
(II)	slower	(II) faster
(III)	fixed count, either increasing or decreasing	(III) any count sequence is possible.
(IV)	decoding error	(IV) no decoding error.

→ Static RAM → cache (no refreshment)
Dynamic RAM → main memory (refreshment)

→ MOD M followed by MOD N counter.
total no of states = MN
counter is called MOD MN counter.

→ Binary counter $N = 2^n$ N = total no of states.
Non Binary counter $N < 2^n$
n = no of FF.

→ Hazards are temporary problems.

→ MUX can be used to implement all digital circuit like adder etc.

→ Decoder (lower to higher) → 3 bit to octal.

→ Base converters are decoder & encoder.

→ XOR is symmetric func^n.

→ Hamming code. (FEC : Forward Error correcting)

Notation (7, 4) indicates

4 data bits, 3 bit for detecting & correcting error

4 message bit + 3 redundant bit = 7 bit Hamming code

$(m > r)$

$2^r \geq m + r + 1$

m = data bits

r = redundant bits / parity bit

→ use option to solve.

→ IEEE standard floating point representation.

(a) Single precision

1 bit	8 bit	23 bit
S	Exponent	Mantissa

Exponent excess - 127

(b) Double precision.

1 bit	11 bit	52 bit
S	Exponent	Mantissa

Exponent excess - 1023

$N = M \times R^e$ eg: $0.00726 \rightarrow 7.26 \times 10^{-3}$

M = mantissa

e = exponent

R = 10

→ Representation of number

→ Fixed point → sign magnitude
→ one's complement
→ two's complement

→ Floating point → IEEE standard.